

Bash Scripting

Guia Prático de Sintaxe

Foco em sintaxe e variações · Direto ao ponto · Linux

Variáveis	Arrays	Strings
Condicionais	Loops	Funções
I/O	Erros	Regex

1. Estrutura Básica de um Script

```
#!/bin/bash
# Isso é um comentário
```

```
echo "Hello, World!"
```

- **#!/bin/bash** — shebang: define o interpretador
- Torne executável: `chmod +x script.sh`
- Execute: `./script.sh` ou `bash script.sh`

2. Variáveis

Declaração e uso

```
nome="João"           # Sem espaços ao redor do =
echo $nome            # Acesso simples
echo "${nome}"        # Acesso com chaves (recomendado)
echo "Olá, ${nome}!"  # Dentro de string
```

Variáveis especiais

\$HOME	# Home do usuário	\$USER	# Usuário atual
\$PWD	# Diretório atual	\$PATH	# Caminhos do sistema
\$\$	# PID do processo	\$?	# Código de saída
\$#	# N° de argumentos	\$@	# Todos os argumentos (lista)
\$*	# Todos os argumentos	\$0	# Nome do script
\$1 \$2	# Argumentos posicionais		

declare — tipos explícitos

```
declare -i numero=5      # Inteiro
declare -r CONST=10      # Somente leitura (readonly)
declare -a arr            # Array indexado
declare -A mapa           # Array associativo
declare -l texto="OLA"   # Força lowercase → "ola"
declare -u texto="ola"   # Força uppercase → "OLA"
local var="aqui"         # Escopo local (dentro de funções)
```

3. Strings

Aspas: simples vs duplas

```
nome="mundo"
echo 'Olá $nome'    # → Olá $nome      (sem interpolação)
echo "Olá $nome"    # → Olá mundo      (com interpolação)
echo Olá $nome      # → Olá mundo      (sem aspas: igual a duplas)
```

Manipulação de strings

```
texto="Hello, World"
echo ${#texto}      # Tamanho: 12
echo ${texto:7}     # Substring a partir de 7: "World"
echo ${texto:7:3}   # Substr de 7, 3 chars: "Wor"
echo ${texto/World/Bash} # Substitui 1ª ocorrência: "Hello, Bash"
echo ${texto//l/L}  # Substitui todas: "HeLLo, WorLd"
echo ${texto^^}     # Maiúsculas: "HELLO, WORLD"
echo ${texto,,}     # Minúsculas: "hello, world"
echo ${texto^}      # Primeira letra maiúscula
```

Valores padrão

```
var=""
echo ${var:-"padrão"}      # Usa "padrão" se var vazia/não definida
echo ${var:="padrão"}      # Atribui e usa "padrão" se vazia/não definida
echo ${var:? "Erro!"}       # Exibe erro e sai se vazia/não definida
echo ${var:+ "tem valor"}   # Retorna "tem valor" só se var definida
```

4. Arrays

Array indexado

```
frutas=("maçã" "banana" "uva")  # Declaração
frutas[3]="kiwi"                 # Adicionar por índice

echo ${frutas[0]}                # maçã (índice começa em 0)
echo ${frutas[@]}                # Todos os elementos
echo ${#frutas[@]}               # Quantidade de elementos
echo ${!frutas[@]}               # Todos os índices

frutas+=("manga")                # Append
unset frutas[1]                  # Remover elemento
echo ${frutas[@]:1:2}            # Fatia: 2 elementos a partir do índice 1
```

Array associativo (dicionário)

```
declare -A pessoa
pessoa["nome"]="Ana"
pessoa["idade"]="30"

echo ${pessoa["nome"]}           # Ana
echo ${!pessoa[@]}               # Todas as chaves
echo ${pessoa[@]}                # Todos os valores
```

Iterando arrays

```
for item in "${frutas[@]}"; do
    echo "$item"
done

# Com índice
for i in "${!frutas[@]}"; do
    echo "$i: ${frutas[$i]}"
done
```

5. Aritmética

```
a=10; b=3

# (( )) — recomendado para inteiros
echo $(( a + b ))    # 13      echo $(( a - b ))    # 7
echo $(( a * b ))    # 30      echo $(( a / b ))    # 3 (inteiro)
echo $(( a % b ))    # 1       echo $(( a ** b ))   # 1000

# Atribuição
(( a++ ))    (( a-- ))    (( a += 5 ))    (( a *= 2 ))

# bc — para decimais
echo "scale=2; 10/3" | bc    # 3.33
echo "sqrt(16)" | bc -l      # 4.000000...
```

6. Condicionais

if / elif / else

```
if [ condição ]; then
    # comandos
elif [ outra_condição ]; then
    # comandos
else
    # comandos
fi
```

Operadores de comparação — Números

```
[ $a -eq $b ]      # igual          [ $a -ne $b ]      # diferente
[ $a -lt $b ]      # menor que     [ $a -le $b ]      # menor ou igual
[ $a -gt $b ]      # maior que     [ $a -ge $b ]      # maior ou igual
```

Operadores de comparação — Strings

```
[ "$s1" = "$s2" ]      # igual
[ "$s1" != "$s2" ]      # diferente
[ -z "$s1" ]            # string vazia
[ -n "$s1" ]            # string não vazia
[[ "$s1" < "$s2" ]]      # comparação lexicográfica
[[ "$s1" =~ ^[0-9]+$ ]]  # match com regex (só em [[]])
```

Operadores de arquivo

```
[ -e arq ]      # existe          [ -f arq ]      # é arquivo regular
[ -d arq ]      # é diretório     [ -r arq ]      # permissão leitura
[ -w arq ]      # perm. escrita   [ -x arq ]      # perm. execução
[ -s arq ]      # existe e !vazio [ -L arq ]      # é link simbólico
```

[] vs [[]] vs (())

```
# [ ]      – POSIX, funciona em sh, mais restrito
[ $a -gt 0 ] && echo "positivo"

# [[ ]] – Bash, mais poderoso (regex, sem aspas obrigatórias)
[[ $nome == "Ana" ]] && echo "é Ana"
[[ $texto =~ ^[A-Z] ]] && echo "começa com maiúscula"

# (( )) – aritmética, retorna 0=true se resultado != 0
(( a > 0 )) && echo "positivo"
```

case

```
case $variavel in
    "opcao1")
        echo "opção 1" ;;
    "opcao2" | "opcao3")    # múltiplos valores
        echo "opção 2 ou 3" ;;
    [0-9]*)                # padrão glob
        echo "começa com número" ;;
    *)                      # default
        echo "outro" ;;
esac
```

7. Loops

for — variações

```
# Lista de valores
for item in a b c d; do echo $item; done

# Intervalo
for i in {1..10}; do echo $i; done
for i in {0..20..2}; do echo $i; done    # de 2 em 2

# Estilo C
for (( i=0; i<5; i++ )); do echo $i; done

# Arquivos
for arq in /tmp/*.log; do echo "Processando: $arq"; done

# Saída de comando
for linha in $(cat arquivo.txt); do echo "$linha"; done
```

while e until

```
# while – executa ENQUANTO condição for verdadeira
contador=0
while [ $contador -lt 5 ]; do
    echo $contador; (( contador++ ))
done

# Ler arquivo linha a linha (forma correta)
while IFS= read -r linha; do
    echo "$linha"
done < arquivo.txt

# until – executa ATÉ condição ser verdadeira
n=0
until [ $n -ge 5 ]; do echo $n; (( n++ )); done
```

Controle de loop

```
for i in {1..10}; do
    [ $i -eq 3 ] && continue    # pula para próxima iteração
    [ $i -eq 7 ] && break       # sai do loop
    echo $i
done
```

8. Funções

```
# Declaração (duas formas equivalentes)
funcao() { echo "Olá da função"; }
function funcao { echo "Olá da função"; }

# Parâmetros, local e return
saudacao() {
    local nome=$1          # $1, $2... são os argumentos
    local sobrenome=$2
    echo "Olá, $nome $sobrenome!"
    return 0               # 0=sucesso, 1-255=erro
}
saudacao "Ana" "Silva"    # Chamada
echo $?                   # Código de retorno

# Retornar um valor real: use echo + $( )
calcular() { echo $(( $1 + $2 )); }
resultado=$(calcular 3 5)
echo $resultado           # 8
```

9. Entrada e Saída

read — leitura de input

```
read variavel          # Lê linha inteira
read -p "Nome: " nome  # Com prompt
read -s senha          # Sem exibir (silencioso)
read -t 5 resposta     # Timeout de 5 segundos
read -n 1 tecla        # Lê apenas 1 caractere
read -a array          # Lê em array
```

echo e printf

```
echo "texto"           # Com newline no final
echo -n "texto"        # Sem newline
echo -e "a
b■c"      # Interpreta escapes (
=newline, ■=tab)

printf "%-10s %5d"
" "Ana" 30    # Formatação com alinhamento
printf "%0.2f"
" 3.14159     # Float com 2 casas decimais
```

Redirecionamento

```
comando > arq.txt      # stdout → arquivo (sobrescreve)
comando >> arq.txt     # stdout → arquivo (acrescenta)
comando < arq.txt      # stdin ← arquivo
comando 2> erros.txt   # stderr → arquivo
comando 2>&1           # stderr → stdout
comando &> tudo.txt     # stdout e stderr → arquivo
comando > /dev/null 2>&1 # descarta toda saída
```

Here-doc e Here-string

```
# Here-doc: bloco de texto como stdin
cat <<EOF
Linha com $variavel interpolada
EOF

cat <<'EOF'           # Sem interpolação
Linha com $variavel literal
EOF

# Here-string: string como stdin
grep "padrão" <<< "minha string de teste"
```

10. Tratamento de Erros

Opções de segurança

```
set -e          # Sai ao primeiro erro
set -u          # Erro em variável não definida
set -o pipefail # Erro em qualquer parte de um pipe
set -x          # Debug: mostra cada comando antes de executar

# Forma curta (combinação recomendada)
set -euo pipefail
```

Códigos de saída e trap

```
# $? → 0=sucesso, qualquer outro valor=falha
```

```

minha_funcao() {
    if [ ! -f "$1" ]; then
        echo "Arquivo não encontrado" >&2    # Erro → stderr
        return 1
    fi
    return 0
}

# trap — captura de sinais
trap "rm -f /tmp/temp_file" EXIT          # Limpa ao sair
trap "echo 'Interrompido!'; exit 1" SIGINT # Ctrl+C

```

11. grep, sed e awk

grep

```

grep "padrão" arq.txt          # Busca básica
grep -i "padrão" arq.txt       # Ignora maiúsculas/minúsculas
grep -n "padrão" arq.txt       # Mostra número de linha
grep -v "padrão" arq.txt       # Inverte (o que NÃO casa)
grep -r "padrão" /dir/         # Recursivo em diretório
grep -E "regex+" arq.txt       # Regex estendida (ERE)
grep -P "\d+" arq.txt          # Regex Perl

```

sed — substituição em stream

```

sed 's/antigo/novo/' arq.txt    # Substitui 1ª ocorrência/linha
sed 's/antigo/novo/g' arq.txt   # Substitui todas (flag g)
sed 's/antigo/novo/gi' arq.txt  # Todas, case-insensitive
sed -i 's/antigo/novo/g' arq.txt # Editar arquivo no lugar
sed '2d' arq.txt                # Deleta linha 2
sed '/padrão/d' arq.txt          # Deleta linhas com padrão
sed -n '3,7p' arq.txt           # Imprime linhas 3 a 7

```

awk — processamento de colunas

```

awk '{print $1}' arq.txt        # 1ª coluna
awk '{print $1, $3}' arq.txt    # 1ª e 3ª coluna
awk -F: '{print $1}' /etc/passwd # Separador customizado
awk 'NR==3' arq.txt             # Imprime linha 3
awk 'NR>=2 && NR<=5' arq.txt    # Linhas 2 a 5
awk '{sum += $1} END {print sum}' # Soma coluna 1
awk '/padrão/ {print $0}' arq.txt # Filtra por padrão
awk '{print NR, $0}' arq.txt    # Adiciona número de linha

```

12. Referência Rápida de Sintaxe

<code>\$var / \${var}</code>	Acessa variável
<code>\$(comando)</code>	Substituição de comando
<code>\$((expr))</code>	Aritmética inteira
<code>\${array[@]}</code>	Todos os elementos do array
<code>\${#array[@]}</code>	Tamanho do array
<code>\${#string}</code>	Tamanho da string
<code>\${string:ini:tam}</code>	Substring
<code>\${string/velho/novo}</code>	Substituição em string

<code>\${var:-padrão}</code>	Valor padrão se vazio
<code>((a > b))</code>	Teste numérico
<code>[[condição]]</code>	Teste com strings/arquivos (Bash)
<code>\$1 \$2 ... \$#</code>	Argumentos posicionais / contagem
<code>\$?</code>	Código de saída do último comando
<code>\$\$</code>	PID do processo atual
<code>2>arq / >&2</code>	Redirecionar stderr
<code>> /dev/null 2>&1</code>	Descartar toda saída
<code>set -euo pipefail</code>	Modo seguro recomendado